

Phase 3 Report — RL-Only Baseline (RecurrentPPO)

MS thesis: ICS-NAD-Gym + LLM-shaped RL for ICS intrusion detection

Generated 2026-07-08 · Trains on ICSNADGym (Phase 2) over the harmonized Parquet corpus (Phase 1) · Pipeline: `src/rl/{train_recurrent_ppo, evaluate_baseline}.py`

Why this report exists. This closes arm (b) of the eventual 3-way comparison (supervised baselines vs. RL-only vs. RL+LLM): a RecurrentPPO policy trained on ICSNADGym's base reward, with no LLM shaping. Getting here required diagnosing and fixing a reproducible native crash in the training stack — documented in full below, since it materially changed the run configuration and is worth knowing about for any later phase that also trains on this GPU.

1. Executive Summary

501,760

timesteps trained

0.952

final explained_variance

0.950

val F1 (RecurrentPPO)

4

crash diagnoses before a
clean run

RecurrentPPO (`sb3-contrib`, `MlpLstmPolicy`) trained cleanly for 501,760 timesteps on CPU after diagnosing a reproducible CUDA-related segfault (Section 3). The trained policy clearly outperforms the naive baselines on detection quality (F1 0.950 on val, 0.752 on test — both far above `always_ALERT`'s 0.745/0.623) while approaching but not matching the oracle ceiling, which is the expected, healthy outcome for a first RL-only baseline.

2. Training Setup

Parameter	Value
Algorithm	RecurrentPPO (<code>sb3-contrib</code>), <code>MlpLstmPolicy</code>
Total timesteps	501,760
Parallel envs	8 (<code>SubprocVecEnv</code> , <code>start_method="spawn"</code>)
Episode length	1,024 flows
n_steps / batch_size	256 / 256
Learning rate / gamma	3e-4 / 0.99
Device	CPU (see Section 3 — not the original plan, diagnosed necessity)
Split	train (per Phase 1 file-level/intra-file assignment)
Reward	Base table only (Phase 2) — no LLM shaping (that's Phase 5/6)

3. Bug Investigation: CUDA/cuDNN-LSTM Segfault

Summary: training crashed with SIGSEGV (exit 139) three times in a row, at three different, non-deterministic step counts, before being correctly diagnosed as a CUDA-specific issue and fixed by switching the policy network to CPU.

3.1 Timeline of the investigation

Attempt	Configuration	Crashed at	Hypothesis tested
1	SubprocVecEnv , fork (default), CUDA	208,896 steps	—
2	SubprocVecEnv , spawn , CUDA	18,432 steps	Fork-unsafety in polars' internal Rust thread pools — ruled out (spawn didn't help, and crashed <i>earlier</i>)
3	SubprocVecEnv , spawn , CUDA, machine confirmed idle	120,832 steps	Resource contention with a concurrent unrelated job (NASimEmu-enhanced) on the same machine — ruled out (still crashed with the machine idle)
4	DummyVecEnv (single-process, no multiprocessing), CUDA	116,736 steps	Multiprocessing/fork/spawn as root cause — ruled out (crashed even single-process)
5	DummyVecEnv , CPU	— completed 200,704 steps cleanly —	Confirmed: CUDA-specific
6 (final)	SubprocVecEnv , spawn , CPU	— completed all 501,760 steps —	Fix validated at full scale

3.2 Root cause and fix

The crash is isolated to CUDA execution of the recurrent (LSTM) policy on this machine's PyTorch 2.12.1+cu130 stack — most likely a cuDNN RNN-kernel compatibility issue given the bleeding-edge library/driver versions involved, though the exact native fault couldn't be inspected directly (this environment has no `dmesg /kernel-log` read permission). Rather than chase a library-version fix blind, the pragmatic resolution was to run this policy on CPU: the network itself is small (MLP + single-layer LSTM over a 55-dim observation), doesn't benefit meaningfully from GPU acceleration, and the RTX 4090 is reserved for the LLM component in later phases anyway — so CPU training carries no real downside here.

Two secondary fixes were also applied and kept regardless of the CUDA finding, since they're correct on their own merits: `start_method="spawn"` for `SubprocVecEnv` (fork is unsafe with polars' internal thread pools in general, even though it wasn't this crash's cause), and a bounded per-process LRU file cache in `ICSNADEmulator.get_cached_file()` (avoids redundant full-file Parquet re-reads on every episode reset, capped at 5 files to bound memory growth).

4. Convergence

Timestep	Entropy loss	Explained variance	Loss
2,048	-1.100	-0.005	265.0
45,056	-0.742	0.602	153.0
86,016	-0.638	0.649	79.1
126,976	-0.426	0.846	750.0
208,896	-0.420	0.927	677.0
290,816	-0.330	0.949	871.0
372,736	-0.190	0.933	1,240.0
454,656	-0.222	0.914	207.0
501,760 (final)	-0.133	0.952	15.9

Entropy loss trends steadily toward zero (policy becoming confident/less random) and explained variance climbs from ~0 to 0.95 (value function tracking returns well) — both healthy, expected convergence signatures. Per-iteration loss is noisy (typical for PPO with a relatively small batch size) but the trend and final low value are consistent with a converged policy.

5. Evaluation vs. Sanity-Floor Policies

15 episodes per policy, episode_length=1024, seed=123, deterministic RL policy actions:

5.1 Validation split

Policy	Reward	Std	Precision	Recall	F1
always_ALLOW	-10,435.67	8,468.84	0.000	0.000	0.000
always_ALERT	3,440.00	5,042.37	0.593	1.000	0.745
RecurrentPPO	2,939.13	2,862.36	0.920	0.983	0.950
oracle	5,610.20	2,815.69	1.000	1.000	1.000

5.2 Test split

Policy	Reward	Std	Precision	Recall	F1
always_ALLOW	-10,185.20	7,415.20	0.000	0.000	0.000
always_ALERT	1,728.33	4,146.77	0.452	1.000	0.623
RecurrentPPO	286.60	3,871.14	0.684	0.835	0.752
oracle	3,839.47	2,295.83	1.000	1.000	1.000

RecurrentPPO clearly beats every naive baseline on F1 at both splits (val: 0.950 vs 0.745; test: 0.752 vs 0.623), and closes most of the gap to the oracle ceiling on validation. The generalization gap between val (0.950) and test (0.752) is real and worth tracking — plausibly related to the per-attack-type validation coverage gap already documented in Phase 1 (several attack types have train+test only, no val file, so the test split carries more novel-to-the-model traffic than val does).

5.3 An important, non-obvious finding: F1 and reward can disagree

On both splits, `RecurrentPPO` has a *higher F1* than `always_ALERT` but a *lower total reward* (val: 2,939 vs 3,440; test: 287 vs 1,728). This isn't a bug — it's the reward design doing exactly what it was built to do. `always_ALERT` never triggers `BLOCK`, so it never pays the steeper false-positive cost (−10 plus an ongoing −3/step quarantine penalty) that a more decisive policy risks when it blocks a benign flow by mistake. `RecurrentPPO` achieves better detection quality (higher precision and F1) partly by using `BLOCK` more, which is riskier per-mistake even though it's net-better at getting the classification right.

This is worth stating plainly as a research finding: **a plain classification metric like F1 is not the same objective as the ICS-specific asymmetric operational cost function**, and optimizing one doesn't automatically optimize the other. That divergence is precisely the motivation for framing this as a sequential RL problem with a cost-aware reward instead of a static supervised classifier — Phase 4's reproduction of the paper's 10 supervised baselines will make this comparison concrete.

6. Files Created

Module	Purpose
<code>train_recurrent_ppo.py</code>	RecurrentPPO training driver: vectorized envs, tensorboard logging, checkpointing
<code>evaluate_baseline.py</code>	Evaluates a trained model + baseline policies on val/test, reports reward and precision/recall/F1

Outputs: `experiments/recurrentppo_baseline_v1/final_model.zip`, 10 intermediate checkpoints (every 50k steps), tensorboard logs under `experiments/tensorboard/`.

7. Next Steps

This closes Phase 3 — arm (b) of the 3-way comparison. Phase 4 reproduces the paper's 10 supervised ML/DL baseline classifiers (`external/Code-for-ICS-NAD-Dataset/Train_All_in_One_czh.ipynb`) on the exact same harmonized Parquet corpus and file-level split used here, giving arm (a) — the fair, apples-to-apples comparison point the paper's own reported numbers can't provide directly, since they used a different (and per Phase 1's findings, less rigorously audited) split.